

Міністерство освіти і науки України
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
“КИЄВО-МОГИЛЯНСЬКА АКАДЕМІЯ”
Кафедра мультимедійних систем
факультету інформатики

Трасування променів у воксельному просторі

Текстова частина до курсової роботи
за спеціальністю “Інженерія програмного забезпечення”

Керівник курсової роботи
старший викладач,
Борозенний С. О.

(підпис)

“ ____ ” _____ 2026 р.

Виконав студент Гнутов О. В.

“ ____ ” _____ 2026 р.

Київ 2026

Міністерство освіти і науки України
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
“КИЄВО-МОГИЛЯНСЬКА АКАДЕМІЯ”
Кафедра мультимедійних систем
факультету інформатики

ЗАТВЕРДЖУЮ
Зав. кафедри мультимедійних систем,
доцент, кандидат наук,
_____ Жежерун О.П.
(підпис)
“ ____ ” _____ 2025 р.

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ
на курсову роботу
студенту Гнутову Олександру 3 курсу
факультету інформатики

ТЕМА: Трасування променів у воксельному просторі

Зміст ТЧ до курсової роботи:

Індивідуальне завдання

Вступ

1. Аналіз предметної області та постановка задачі
2. Методи та особливості реалізації трасування у воксельному просторі
3. Реалізація алгоритму трасування променів у воксельному просторі

Висновок

Список використаних джерел

Додатки

Дата видачі “ ____ ” _____ 2026 р.

Керівник _____ Завдання отримав: _____

КАЛЕНДАРНИЙ ПЛАН ВИКОНАННЯ КУРСОВОЇ РОБОТИ

№ з/п	ПЕРЕЛІК РОБІТ	Термін виконання	Дата ознайомлення наукового керівника	Підпис наукового керівника	Примітки
1.	Вибір теми, затвердження її на засіданні кафедри та закріплення наукового керівника. Узгодження календарного графіка підготовки курсової роботи. Ознайомлення студента з критеріями оцінювання курсової роботи	14 грудня 2025			
2.	Вивчення джерел літератури, матеріалів архівів, періодичних видань, збір та узагальнення фактів, даних	15 грудня 2025 — 31 січня 2026			
3.	Складання плану курсової роботи та узгодження з науковим керівником	31 січня 2026			
4.	Написання розділів роботи	1 лютого — 10 березня 2026			
5.	Проміжний контроль виконання роботи	1 лютого 2026			
6.	Написання курсової роботи в цілому, ознайомлення з її першим варіантом наукового керівника	11 лютого — 10 березня 2026			
	Розділ 1 (постановка проблеми, теоретичні основи, огляд літературних джерел)	25 лютого 2026			
	Розділ 2 (аналітично-дослідницька частина)	1 березня 2026			
	Розділ 3 (проєктно-рекомендаційна частина)	10 березня 2026			
7.	Повне завершення написання курсової роботи, оформлення її згідно з вимогами й подання на відгук науковому керівнику	17 березня 2026			
8.	Подання курсової роботи для перевірки письмових робіт студентів НаУ-КМА на відповідність вимогам академічної доброчесності	24 березня 2026			
9.	Публічний захист курсової роботи перед екзаменаційною комісією	6–8 квітня 2026			

Графік узгоджено “_____” _____ 2025 р.

Науковий керівник: Борозенний Сергій Олександрович

Виконавець курсової роботи: Гнутов Олександр Володимирович

Зміст

АНОТАЦІЯ	1
ВСТУП	2
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ	4
1.1 Поняття вокселя та воксельних структур даних	4
1.2 Основи трасування променів	5
1.2.1 Основні проблеми реалізації алгоритму трасування про- менів	7
1.3 Методи трасування променів у дискретному просторі	8
1.3.1 Пряме та інверсивне мапування	10
1.4 Проблеми, що вирішує воксельне представлення	10
1.5 Постановка задачі	12
2 МЕТОДИ ТА ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ ТРАСУВАННЯ У ВОКСЕЛЬНОМУ ПРОСТОРІ	14
2.1 Алгоритм DDA	14
2.2 Fast Voxel Traversal Algorithm (Amanatides and Woo)	15
2.3 Структури даних для трасування на GPU	18
2.3.1 Лінійна BVH	18

2.3.2	3D текстура	20
2.3.3	3D MIP-map як аналог octree	20
2.3.4	Загальний воксельний об'єм (World Volume)	21
2.4	Оптимізації	22
3	РЕАЛІЗАЦІЯ АЛГОРИТМУ ТРАСУВАННЯ ПРОМЕНІВ У ВОКСЕЛЬНОМУ ПРОСТОРІ	23
3.1	Створення базових структур даних та матеріалів на базі рушія Bevy мовою Rust	23
3.1.1	Архітектура системи плагінів матеріалів	23
3.1.2	Організація модульної структури проєкту	24
3.1.3	Реалізація структури VoxelRayTracedMaterial	25
3.1.4	Система компонентів для воксельних об'єктів	27
3.1.5	Система World Volume	27
3.1.6	Завантаження воксельних даних з формату 2D слайсів	28
3.2	Реалізація повного пайплайну відкладеного рендерингу для тра- сування у воксельному просторі	29
3.2.1	Теоретичні засади відкладеного рендерингу	29
3.2.2	Створення G-buffer attachments	30
3.3	Написання алгоритму трасування шейдерною мовою WGSL . .	32
	ВИСНОВКИ	33
	СПИСОК ЛІТЕРАТУРИ	34
	ДОДАТКИ	36

АНОТАЦІЯ

ВСТУП

Сучасні комп'ютери, і особливо їх графічні процесори, можуть обробляти все більше даних за більш короткий, що відкриває нові можливості в галузі комп'ютерної графіки. Однак, більшість відеоігор та інших графічних застосунків досі використовує більш продуктивний, але менш фізично-коректний і реалістичний метод рендерингу — растеризацію, замість більш ресурсно-затратної технології — трасування променів.

Растеризація — це процес трансформації векторних даних (зазвичай множини трикутників) в растрове зображення, яке графічний процесор по пікселю відмальовує на екрані. В той же час трасування променів (*англ. ray tracing*) — це зовсім інший підхід, який полягає в отриманні інформації про об'єкти рендерингу через кидання променю з кожного пікселя на їх поверхню. Із самого означення зрозуміло, що наївна реалізація алгоритму трасування променів досить ресурсно-затратна, і тому вимагає оптимізацій для того, щоб можна було використати цей алгоритм на практиці.

Метою цієї роботи є розглянути основні поняття, що стосуються трасування променів, в тому числі воксельного, різні методи оптимізації рей трейсингу, їх поєднання з методом трасування у воксельному просторі, а також переваги та недоліки цього методу.

Дана робота ставить на меті виконання наступних завдань:

- ◇ Аналіз та порівняння різних методів трасування у дискретному та неперервному просторі;
- ◇ Реалізація алгоритму трасування воксельних об'ємів і створення проміжних даних для подальшого рендерингу;
- ◇ Реалізація пост-процесингового алгоритму відкладеного (*англ. deferred*) трасування у воксельному просторі з використанням загального

воксельного об'єму, створеного на основі інших об'єктів 3D-сцени;

- ◇ Дослідження додаткових методів оптимізації трасування воксельної сітки;
- ◇ Створення кінцевого застосунку, що демонструє всі наведені вище алгоритми рендерингу на прикладі воксельних 3D моделей.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Поняття вокселя та воксельних структур даних

Воксель (англ. **volume** + **pixel**) — одиничний елемент *воксельного* простору, що являє собою решітку з елементів фіксованого розміру. У тривимірному просторі є аналогом пікселя у двовимірному. У процесі трасування променів він грає роль як *візуального елемента*, оскільки 3D моделі у даному проєкті будуть самі будуть складатися з фіксованої сітки вокселів, так і *логічного елемента*, так як воксель є частиною загального воксельного об'єма (World Volume), який буде використовуватися у відкладеному рендерингу. [1]

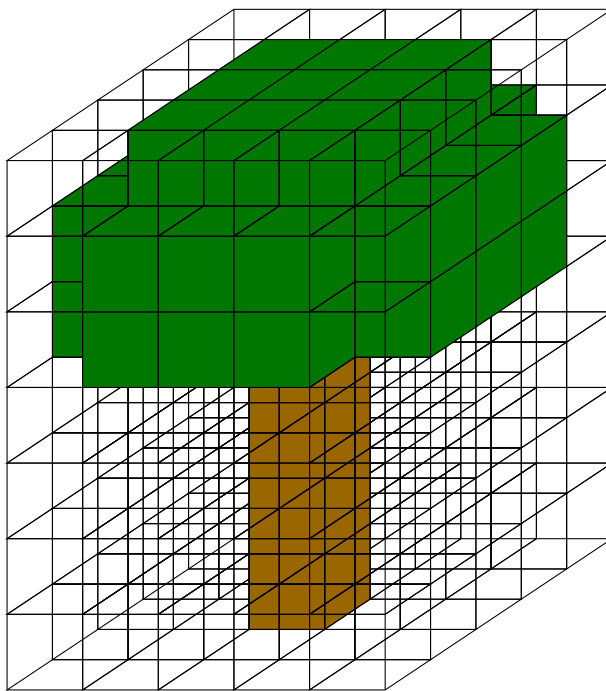


Рис. 1.1:
Воксельна модель дерева, представлена у вигляді тривимірного масиву вокселів.

Найпростішою формою представлення воксельного об'єму є однорідний **тривимірний масив**. Це масив розміру $N \times M \times P$, де N, M, P — це розміри обмежувальної коробки воксельної моделі. У реалізації в шейдерній мові він може бути представлений у вигляді лінійного масиву, індекс якого за координатами x, y, z буде рахуватися за формулою: $x + y \times N + z \times N \times M$; або ж у вигляді 3D-текстури, з якою можна працювати за допомогою вбудованої функції `textureLoad(x, y, z, L)`, де L — це MIP-рівень, який ми пізніше використаємо для подальшої оптимізації.

Однією із більш просунутих структур даних для зберігання воксельного об'єму є **октодерево** (*англ.* **octree**) — дерево, в якому кожен вузол має рівно вісім нащадків і представляє собою кубічний об'єм простору, який розділяється на вісім менших кубів при досягненні певного критерію розбиття; в нашому випадку — якщо цей об'єм рекурсивно містить хоча б один “твердий” воксель. Octree дозволяє ефективно зберігати та обробляти великі об'єми даних, оскільки дозволяє зберігати інформацію лише про ті частини простору, які містять важливі дані, і пропускати порожні області. [2]

Однак, для оптимізації пам'яті використання октодерева на GPU є нелегкою задачею через відсутність у шейдерних мовах вказівників, щоб не зберігати порожні області воксельного об'єму у пам'яті. Проте в цій роботі для зберігання воксельного об'єму буде використана 3D текстура з MIP-рівнями, що дозволить принаймні оптимізувати прохід по воксельному об'єму за допомогою рівнів октодерева.

Вокселізацією називається процес перетворення будь-яких тривимірних даних (сфера, задана формулою, меш 3D моделі) у дискретне (воксельне) представлення. Також вокселізацією можна назвати перетворення воксельних об'ємів, що не вирівняні за воксельною сіткою; наприклад, до яких застосована трансформація (зміщення, поворот, масштаб). Такий процес ще називається мапуванням (*англ.* **mapping**), подібно до мапування у 2D зображеннях.

1.2 Основи трасування променів

Трасування променів — це процес відстеження шляху від уявного ока (камери) через кожен піксель області перегляду (*англ.* **Viewport**) та обчислення

кольору видимого (на який падає промінь) об'єкта, враховуючи навколишнє середовище, джерела світла, фізичні властивості матеріалу об'єкта тощо.

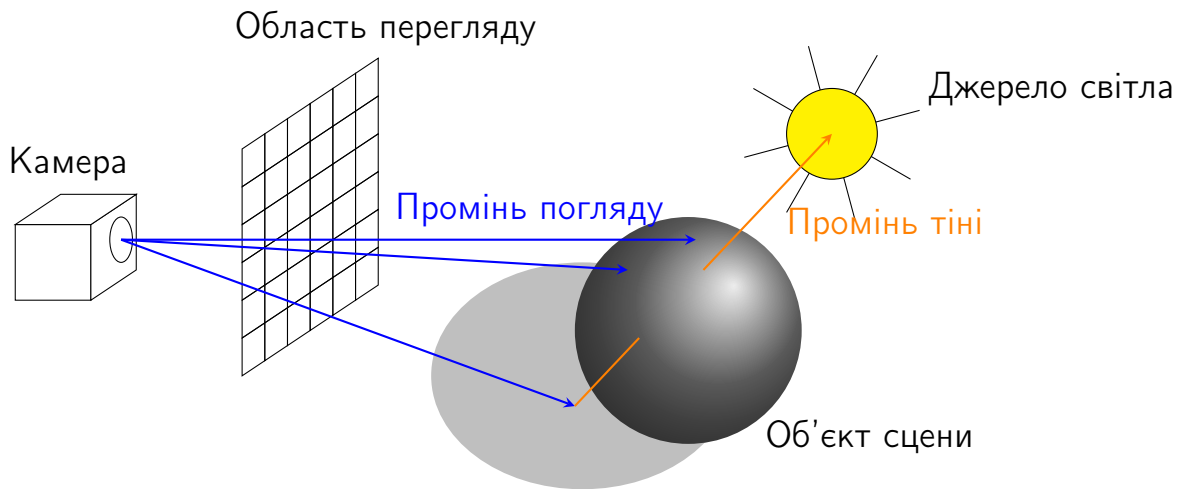


Рис. 1.2:
Діаграма трасування променів.

Загальне трасування променів є потужним методом рендерингу, який дозволяє досягти високої якості зображення шляхом моделювання фізичних явищ, що відбуваються при взаємодії світла з об'єктами. Цей підхід включає в себе не лише базове трасування променів для визначення видимості об'єктів, але й складніші техніки, такі як обчислення тіней, відбиттів та заломлень. Основна ідея загального трасування променів полягає в тому, щоб відстежувати шлях променів світла від джерел до камери, враховуючи всі можливі взаємодії з об'єктами на сцені. Це дозволяє створювати реалістичні зображення, які враховують не лише геометрію об'єктів, але й їх матеріали, освітлення та інші параметри сцени на об'єктів.

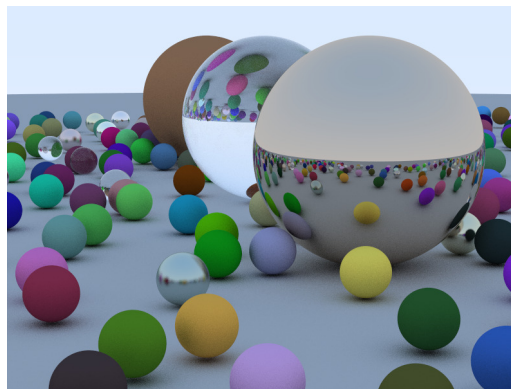


Рис. 1.3:
Приклад трасування променів для сфер на площині[3].

Програмно трасувальник променів кидає промінь від кожного пікселя, який

проходить через кожен об'єкт на сцені за один прохід рендерингу, а потім відображає результат на площині розміром з область перегляду, що являє собою комбінацію двох растеризованих трикутників. Сам *промінь* — це структура даних, яка складається з **точки початку** і **вектору напрямку**. При повторному відбиванню променів від поверхні, точкою початку наступного променя стає точка перетину променя з об'єктом.

1.2.1 Основні проблеми реалізації алгоритму трасування променів

Під час реалізації найвісного алгоритму рей трейсингу, основними проблемами, з якими можна зіткнутися:

1. **“Тіньове акне”** (*англ.* **shadow acne**). Це типова проблема, яка з'являється під час трасування променів для тіней, навколишнього перекриття (*англ.* **ambient occlusion, AO**) та інших видів затінення, коли промінь відбивається від поверхні об'єкта і перетинає сам себе, оскільки його точка початку знаходиться під поверхнею, що зумовлене проблемою точності обчислень чисел з рухомою комою. Простий та ефективний метод вирішення даної проблеми — це додавання невеликого відступу від поверхні об'єкта сцени вздовж нормалі даної поверхні до точки початку променя.
2. **“Екстремальне” тіньове акне**. Воно виникає під час трасування променів у воксельному просторі, коли вокселі вокселізованого об'єкта сцени “вибирають” з-під самого об'єкта, перетинаючи його поверхню, що створює артефакти, подібні до звичайного тіньового акне. Так само, як і для звичайного тіньового акне, можна використати відступ вздовж нормалі, однак для великих розмірів одиничного вокселя це не дасть значного ефекту, а при збільшенні цього відступу точність затінення буде зменшуватися. Тому окрім відступу вздовж нормалі поверхні, точка початку променя додатково зміщується у напрямку самого променя на частку розміру вокселя. Такий зсув гарантує, що початок трасування знаходиться не лише поза геометрією об'єкта, але й поза граничною площиною вокселя, усуваючи ситуацію, коли перший крок DDA-алгоритму

потрапляє в той самий воксель.

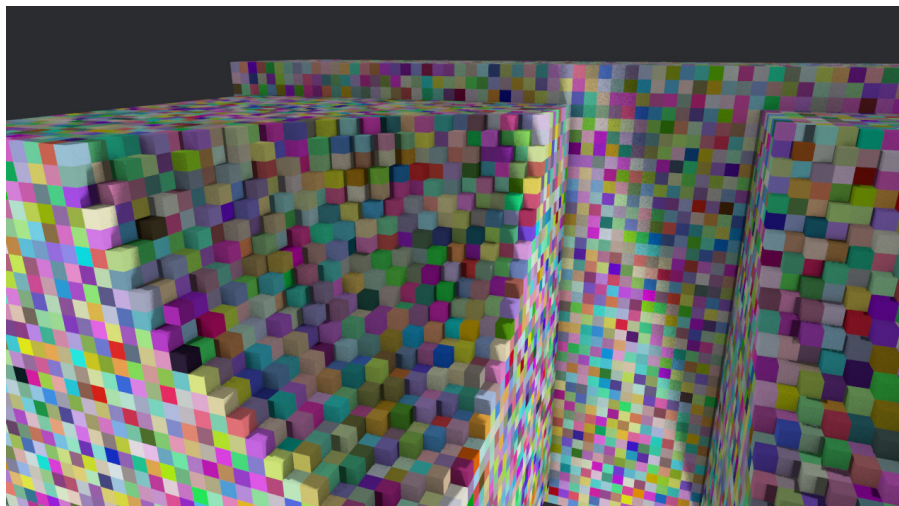


Рис. 1.4:

Екстремальне тіньове акне в орієнтованому воксельному об'ємі.

3. **Перекривання об'єктів.** При неправильній роботі з буфером глибини (Z-буфер), дані попередньо відтрасованих об'єктів перезаписуються незалежно від того, чи об'єкт знаходиться ближче до камери, чи далі. Щоб виправити цю проблему, необхідно на етапі запису в кольоровий буфер перевіряти значення у Z-буфері в даному пікселі і порівнювати його з поточним значенням:

- ◇ якщо воно більше за поточне — об'єкт ближче, записуємо колір і перезаписуємо значення Z-буфера;
- ◇ якщо менше за поточне — об'єкт знаходиться далі від камери, пропускаємо піксель і залишаємо буфер незмінним.

1.3 Методи трасування променів у дискретному просторі

Оскільки трасування променів являє собою важке обчислення, особливо для моделей, що складаються з великої кількості полігонів, а також сцен із великою кількістю об'єктів, існує два методи для оптимізації цього алгоритму: побудова ієрархії обмежувальних об'ємів (*англ.* **BVH**) для об'єктів сцени та розділення простору на дискретні об'єми з вокселів. Детально обидва методи ми розглянемо пізніше. [4]

В даній роботі ми розглянемо та опишемо таку структуру даних, як загальний (світовий) воксельний об'єм (*англ.* **World Voxel Volume**), що являє собою великий ($\approx 1000^3$ вокселів) масив із вокселів, утворений вокселізацією усіх об'єктів на сцені. Він має багато переваг для наших цілей:

- ◇ просте та зручне представлення на GPU за допомогою єдиної 3D текстури;
- ◇ підтримка оптимізації октодерева (через MIP-рівні текстури);
- ◇ відсутня побудова дерева та його балансування;
- ◇ вокселізація у загальний об'єм воксельних 3D моделей (наприклад, у вигляді 2D слайсів), навіть *орієнтованих* (до яких застосована матриця повороту), які ми використовуємо для нашого застосунку, є тривіальною.

В той же час, такий спосіб має переваги і над лінійним представленням воксельних (або вокселізованих) об'єктів сцени в масиві:

- ◇ відсутня потреба у зберіганні додаткових структур даних (таких як BVH) для кожного об'єкту сцени, оскільки інакше проходження по лінійному масиву з багатьох об'єктів, не збалансованих за відстанню, буде більш ресурсозатратним;
- ◇ простота реалізації алгоритму трасування променів, оскільки всі об'єкти сцени зберігаються в єдиному просторі;
- ◇ можливість зменшення використаної пам'яті за рахунок побітового представлення вокселя в загальному об'ємі (1 — твердий воксель, 0 — порожній).

Однак даний метод має і свої недоліки:

- ◇ обмеження сцени розмірами воксельного об'єму; однак для великих процедурних світів може бути використана система чанків;
- ◇ побітове представлення вокселів хоч і зменшує використану пам'ять у рази, світовий об'єм для великої сцени може важити більше 200 MiB, при цьому втрачається інформація про колір вокселя та інші його атрибути, що в свою чергу перешкоджає реалізації більш реалістичних технік в трасуванні променів, таких як глобальне освітлення (*англ.* **Glo-**

bal Illumination, GI);

- ◇ розмір одиничного вокселя — пропорційний використуваній пам'яті GPU та швидкості, але обернено пропорційний точності обчислення.

Тому цей метод в першу чергу орієнтований на швидкодію, простоту реалізації та подібну до фізичної (а не цілком засновану на фізичних явищах) графіку.

1.3.1 Пряме та інверсивне мапування

При мапуванні об'єктів у загальний воксельний об'єм використовуються два основні підходи: **пряме** та **інверсивне** мапування.

Пряме мапування полягає у безпосередньому проходженні всіх елементів об'єкта (вокселів або полігонів) і записі відповідних вокселів у світовий об'єм. Такий підхід простий у реалізації та працює швидше за інверсивне мапування через те, що ітерація відбувається по самих вокселях, однак це спричиняє появу “прогалин” у світовому об'ємі, оскільки, залежно від трансформації об'єкту, мапованих вокселів може бути більше, ніж початкових.

Інверсивне мапування використовує зворотне перетворення: для кожного вокселя світового об'єму визначається, чи потрапляє він всередину об'єкта, шляхом зворотного відображення координат у локальну систему об'єкта. Цей метод дозволяє уникнути пропусків і гарантує заповнення всіх вокселів, що належать об'єкту, особливо при дискретизації з низькою роздільною здатністю.[5]

У межах даної роботи, враховуючи використання воксельного об'єму для трасування променів на GPU, перевага надається інверсивному мапуванню, оскільки воно забезпечує більш рівномірне та компактне заповнення об'єму без прогалин, що критично для трасування променів.

1.4 Проблеми, що вирішує воксельне представлення

Якщо говорити про сучасні ігри та інші графічні застосунки, то вони великою мірою працюють із полігональними 3D-моделями, які в свою чергу є одними

з найбільш важких для обчислення об'єктів трасування променів. Того для полігональних моделей вокселізація і розташування у загальному об'ємі є дуже ефективним методом оптимізації:

- ◇ незалежно від кількості полігонів, точність обчислення все ще буде залежати від розміру одиничного вокселя; полігони впливають лише на швидкість процесу вокселізації;
- ◇ для далеких від камери об'єктів можлива оптимізація за допомогою MIP-рівнів, подібно до рівнів деталізації (*англ. Level of Detail, LOD*) в процесі рендерингу растеризацією.

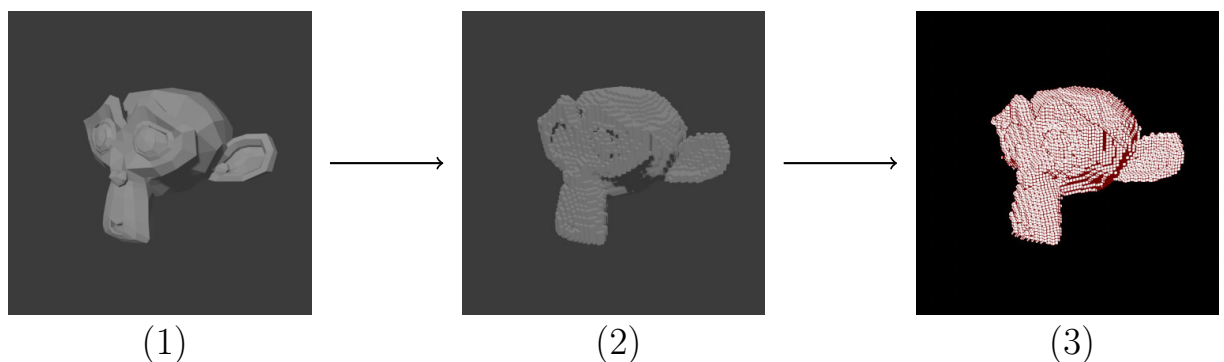


Рис. 1.5:

Вокселізація та мапування полігональної моделі в загальний об'єм:

- 1 — полігональна модель;
- 2 — вокселізована модель;
- 3 — представлення у загальному об'ємі (білий — тверді воксели, чорний — порожні; червоною сіткою виділено окремі воксели).

В нашому випадку, оскільки сама графіка у нашому застосунку є воксельною, представлення всієї сцени у вигляді загального воксельного об'єму все ще дає нам наступні переваги:

- ◇ як і вказано вище, це звільняє нас від потреби власноруч *ефективно* організовувати простір для окремих воксельних моделей, у вигляді збалансованих BVH дерев, складних октодерев тощо, оскільки всі моделі підлягають процесу мапування у загальний об'єм, і доступ до даних всередині них, на відміну від пошуку по дереву, відбувається за константний час за допомогою тривимірного індексу;
- ◇ можливість використання MIP-рівнів 3D текстури для швидкого пропуску великих порожніх областей простору під час трасування, що ана-

логічно до переваг октодерева, але без складності його побудови;

- ◇ єдиний уніфікований простір координат спрощує обчислення взаємодій між об'єктами (тіні, відбиття, ambient occlusion), оскільки всі об'єкти вже знаходяться в одній системі координат без потреби додаткових трансформацій.

1.5 Постановка задачі

Метою даної роботи є розробка та дослідження методу рендерингу тривимірних сцен на основі трасування променів у дискретному воксельному просторі з використанням загального світового воксельного об'єму, орієнтованого на виконання на графічному процесорі. Основний акцент робиться не на повній фізичній коректності освітлення, а на досягненні прийняттого балансу між якістю зображення, швидкодією та простотою реалізації, що є критично важливим для інтерактивних застосунків, зокрема ігрових рушіїв та візуалізацій у реальному часі.

У межах роботи передбачається, що всі об'єкти сцени — як спочатку воксельні, так і полігональні — проходять етап вокселізації та малуються в єдиний світовий воксельний об'єм фіксованої роздільної здатності. Цей об'єм зберігається у вигляді 3D-текстури з MIP-рівнями та використовується як основне джерело даних для алгоритму трасування променів. Таким чином, задача трасування зводиться до ефективного проходження дискретного простору та визначення перетинів променя з твердими вокселями.

В рамках поставленої мети необхідно розв'язати такі основні задачі:

- ◇ спроектувати модель представлення світового воксельного об'єму, придатну для зберігання на GPU, з урахуванням обмежень шейдерних мов та обсягу відеопам'яті;
- ◇ розробити алгоритм малування (вокселізації) окремих об'єктів сцени у спільний дискретний простір, включно з підтримкою трансформацій (зміщення, обертання, масштабування);
- ◇ реалізувати алгоритм трасування променів у воксельному просторі на основі інкрементального проходження (DDA або подібного підходу),

адаптований до дискретної природи даних;

- ◇ дослідити та врахувати характерні числові та геометричні проблеми такого підходу, зокрема явища тіньового та «екстремального» тіньового акне, та запропонувати практичні способи їх усунення;
- ◇ використати MIP-рівні 3D-текстури як наближену форму ієрархії простору для прискорення трасування шляхом пропуску великих порожніх ділянок;
- ◇ забезпечити коректну роботу з буфером глибини для підтримки перекривання об'єктів та поєднання результатів трасування з іншими етапами рендерингу.

Окремою задачею є визначення меж застосовності запропонованого підходу. Зокрема, необхідно врахувати, що розмір одиничного вокселя безпосередньо впливає як на візуальну точність, так і на споживання пам'яті та продуктивність. Тому в роботі приймається припущення, що сцена має обмежені розміри або ж може бути розбита на окремі чанки, які підвантажуються та обробляються незалежно. Також допускається спрощене представлення властивостей матеріалів вокселів (наприклад, без повноцінної підтримки глобального освітлення), якщо це дозволяє суттєво зменшити складність реалізації.

У результаті виконання роботи очікується отримання прототипу рендереру, який:

- ◇ демонструє можливість використання єдиного світового воксельного об'єму як універсальної структури даних для трасування променів;
- ◇ забезпечує стабільну роботу в реальному часі для сцен середньої складності;
- ◇ може бути розширений у майбутньому за рахунок додавання більш складних моделей освітлення, матеріалів або динамічних змін сцени.

2 МЕТОДИ ТА ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ ТРАСУВАННЯ У ВОКСЕЛЬНОМУ ПРОСТОРІ

2.1 Алгоритм DDA

Одним із базових алгоритмів проходження дискретного простору вздовж променя є алгоритм цифрового диференціального аналізатора (*англ.* **Digital Differential Analyzer, DDA**). Спочатку він застосовувався для растеризації ліній на двовимірній сітці, однак згодом був узагальнений на тривимірний випадок і може використовуватися для проходження регулярних воксельних сіток.

У контексті трасування променів у воксельному просторі алгоритм DDA дозволяє ітеруватися вздовж променя з фіксованим кроком, обчислюючи на кожному кроці координати точки на промені та визначаючи відповідний воксель. Таким чином, промінь відстежується послідовно, крок за кроком, а складність алгоритму пропорційна довжині променя та обраному кроку, а не безпосередньо кількості відвіданих вокселів.[6]

Формально, промінь у тривимірному просторі задається параметричним рівнянням:

$$\mathbf{r}(t) = \mathbf{o} + t \cdot \mathbf{d},$$

де $\mathbf{o}$ — точка початку променя, $\mathbf{d}$ — нормалізований вектор напрямку, а $t \geq 0$ — параметр.

Метою алгоритму DDA є визначення послідовності цілих координат (x, y, z) вокселів, через які проходить промінь, шляхом ітеративного крокування вздовж променя і взяття цілих координат відповідно до обчисленої позиції.

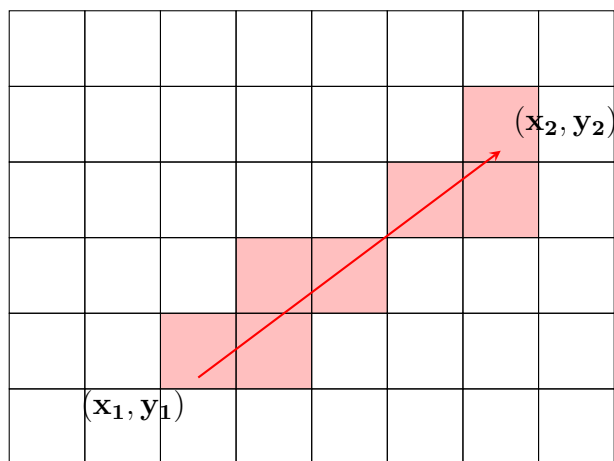


Рис. 2.1:
Ілюстрація роботи алгоритму DDA у двовимірному випадку.

Процес повторюється доти, доки промінь не досягне цільового вокселя або не покине межі воксельного об'єму.

Основною перевагою наївного DDA є його простота реалізації та універсальність. Водночас він є менш ефективним для великих сцен, оскільки виконує багато зайвих обчислень і може пропускати тонкі вокселі або перетинати один воксель декілька разів, якщо крок обраний невдало.

2.2 Fast Voxel Traversal Algorithm (Amanatides and Woo)

Алгоритм швидкого проходження воксельного простору, запропонований Ендрю Ву та Джоном Аманатайдс з університету Торонто, є спеціалізованим та оптимізованим варіантом тривимірного DDA, адаптованим саме для задач трасування променів у воксельних сітках.

Його ключовою особливістю є те, що всі величини, необхідні для проходження, обчислюються один раз перед початком ітерацій, а подальші кроки алгоритму не потребують операцій ділення, що є критично важливим для продуктивності на GPU.[7]

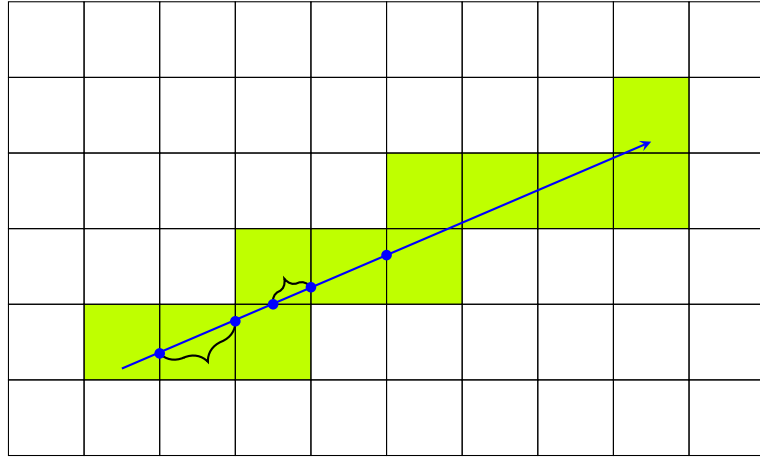


Рис. 2.2:

Проходження променя через послідовність вокселів за алгоритмом Amanatides and Woo.

Алгоритм починається з визначення початкового вокселя, у якому знаходиться точка **o**. Далі для кожної осі i обчислюються:

$$\begin{aligned} \mathbf{step}_i &= \text{sign}(d_i), \\ \mathbf{tDelta}_i &= \frac{s_i}{|d_i|}, \\ \mathbf{tMax}_i &= \begin{cases} (\lfloor o_i \rfloor + 1 - o_i) \cdot \mathbf{tDelta}_i, & d_i > 0 \\ (o_i - \lfloor o_i \rfloor) \cdot \mathbf{tDelta}_i, & d_i < 0 \end{cases} \end{aligned}$$

1. $\mathbf{step}_i$ — напрям руху по осі (+1 або -1);
2. $\mathbf{tDelta}_i$ — приріст параметра t між двома послідовними межами вокселів по осі i , де s_i — розмір вокселя по осі i ;
3. $\mathbf{tMax}_i$ — значення t , при якому промінь уперше перетне межу наступного вокселя вздовж осі i .

Далі виконується основний цикл проходження:

- ◇ На кожному кроці вибирається мінімальне з $\mathbf{tMax}_x$, $\mathbf{tMax}_y$, $\mathbf{tMax}_z$, що визначає, через яку грань вокселя промінь вийде наступним.
- ◇ Відповідний індекс координати збільшується або зменшується згідно $\mathbf{step}_i$.
- ◇ $\mathbf{tMax}_i$ оновлюється: $\mathbf{tMax}_i \leftarrow \mathbf{tMax}_i + \mathbf{tDelta}_i$.
- ◇ Перевіряється вихід за межі об'єму або досягнення цільового вокселя.

Псевдокод алгоритму:

```
loop {
    if t_max_x < t_max_y {
        if t_max_x < t_max_z {
            x += step_x;
            if x == just_out_x {
                return None;
            }
            t_max_x += t_delta_x;
        } else {
            z += step_z;
            if z == just_out_z {
                return None;
            }
            t_max_z += t_delta_z;
        }
    } else {
        if t_max_y < t_max_z {
            y += step_y;
            if y == just_out_y {
                return None;
            }
            t_max_y += t_delta_y;
        } else {
            z += step_z;
            if z == just_out_z {
                return None;
            }
            t_max_z += t_delta_z;
        }
    }
}

let result = load_voxel(x, y, z);
if result.is_some() {
    return result;
}
```

На кожному кроці вибирається мінімальне з $tMax_x$, $tMax_y$, $tMax_z$, що означає, через яку грань вокселя промінь вийде наступним.

У порівнянні з наївним підходом, алгоритм Amanatides and Woo:

- ◇ гарантує відвідування кожного вокселя не більше одного разу;
- ◇ не використовує умовних перевірок перетину з геометрією;
- ◇ має передбачуваний контроль потоку виконання, що добре підходить для SIMD-архітектури GPU.

Завдяки цим властивостям, алгоритм широко використовується для трасування променів у воксельних сценах, особливо на GPU.

Саме цей алгоритм використовується як базовий механізм трасування у подальших розділах роботи.

2.3 Структури даних для трасування на GPU

Ефективність алгоритму трасування променів у воксельному просторі значною мірою залежить не лише від самого алгоритму проходження, а й від обраної структури даних для зберігання сцени. В умовах GPU існують суттєві обмеження, зокрема відсутність вказівників, обмежений стек викликів та висока вартість розгалужень.

2.3.1 Лінійна BVH

Ієрархія обмежувальних об'ємів (*англ.* **Bounding Volume Hierarchy, BVH**) — це класична структура даних для прискорення тестів на перетин променя з геометрією сцени. Основна ідея BVH полягає в побудові ієрархічного дерева обмежувальних об'ємів, де кожен вузол містить обмежувальний об'єм для певної групи примітивів сцени (трикутників, полігонів тощо). Листові вузли містять безпосередньо примітиви, а внутрішні вузли — посилання на нащадків, що дозволяє швидко відсіювати великі частини сцени при трасуванні променів.

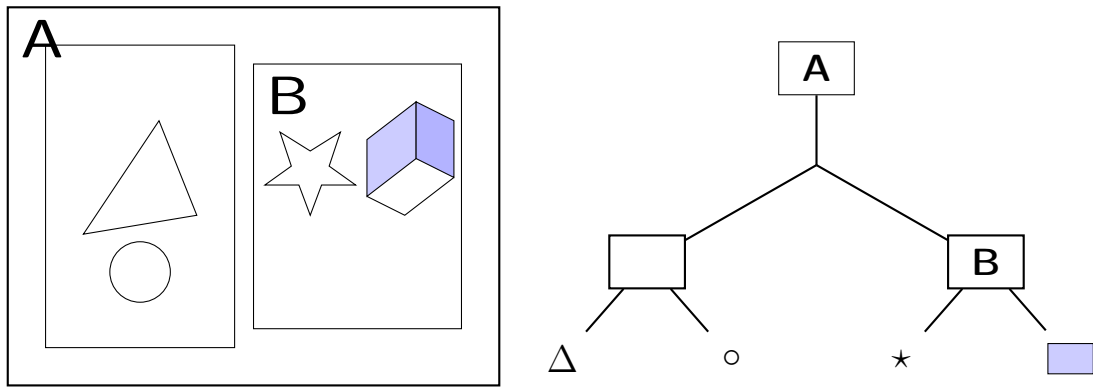


Рис. 2.3:

Схематичне представлення класичної BVH для полігональної сцени.

На GPU класичне дерево з вказівниками використовувати складно через відсутність прямої підтримки динамічних структур і рекурсії у шейдерах. Для вирішення цієї проблеми застосовується **лінійна BVH** (*англ.* **Linear BVH**, **LBVH**), де вузли дерева зберігаються у лінійному масиві. Кожен вузол містить:

- ◇ координати обмежувального об'єму;
- ◇ індекси лівого та правого нащадків у масиві;
- ◇ інформацію про листовий вузол (наприклад, індекси примітивів).

Цей підхід дозволяє обійтись без рекурсії та працювати з детермінованим доступом до пам'яті, що є критично важливим для продуктивності на GPU. Проте, для задач трасування у воксельному просторі використання LBVH є надлишковим. Основні причини:

1. **Вартість побудови та оновлення.** Навіть лінійна BVH потребує додаткових обчислень для сортування і побудови масиву вузлів. Для динамічних сцен або при зміні розташування об'єктів це може значно вплинути на швидкодію.
2. **Пропуск порожніх областей.** LBVH добре працює для полігонів, де більшість простору порожня. У випадку вокселізованої сцени всі активні воксели вже зберігаються у єдиному об'ємі, і пропускати порожні зони можна більш ефективно через MIP-рівні 3D текстури.

Замість LBVH у цій роботі обрана стратегія використання **загального світового воксельного об'єму** (*англ.* **World Voxel Volume**) з MIP-рівнями, що дозволяє:

- ◇ зберігати всю сцену в єдиній 3D текстурі;
- ◇ виконувати швидкий доступ до вокселів за константний час через тривимірний індекс;
- ◇ реалізувати ефективний пропуск порожніх областей аналогічно до октодерева, без необхідності зберігати явну ієрархію з вузлів і вказівників;
- ◇ спростити реалізацію трасування на GPU, уникаючи складної рекурсії та великих масивів LBVH.

Таким чином, LBVH залишається корисною концепцією для полігональної геометрії, проте у контексті воксельного рендерингу її роль значно зменшується, а структура світового об'єму з MIP-рівнями забезпечує подібні переваги, зберігаючи при цьому простоту та ефективність GPU-реалізації.

2.3.2 3D текстура

Найпростішим і водночас найефективнішим способом зберігання воксельного об'єму на GPU є використання 3D текстури. Кожен тексель відповідає одному вокселю та може містити інформацію про:

- ◇ твердість вокселя;
- ◇ колір;
- ◇ ідентифікатор матеріалу.

Перевагою 3D текстур є апаратна підтримка кешування та можливість швидкого довільного доступу за координатами (x, y, z) .

2.3.3 3D MIP-map як аналог octree

MIP-mapping — це техніка текстуровання, яка використовує кілька копій однієї й тієї ж текстури з різним рівнем деталізації. MIP-рівні 3D текстури можуть бути використані як спрощений аналог октодерева. Кожен наступний рівень відповідає простору, де кожен воксель агрегує інформацію з $2 \times 2 \times 2$ вокселів попереднього рівня: якщо воксель твердий, то в даному просторі є хоча б 1 твердий воксель на нижчому рівні.

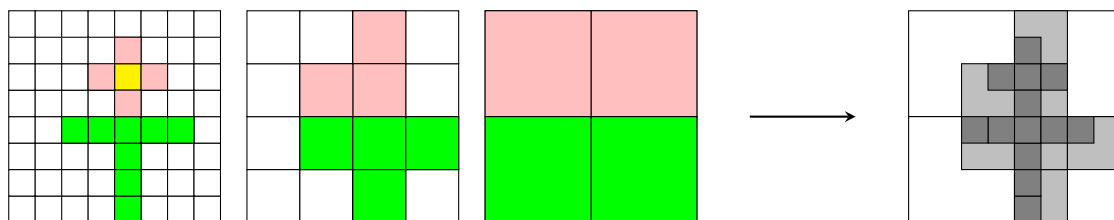


Рис. 2.4:
Відповідність між MIP-рівнями 2D текстури та
двовимірною проєкцією октодереву
(також працює в 3D).

Це дозволяє:

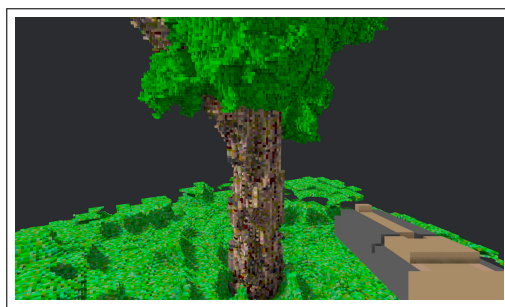
- ◇ швидко перевіряти великі області простору на порожність;
- ◇ виконувати грубий тест перетину на старших рівнях;
- ◇ переходити до точнішого рівня лише за необхідності.

2.3.4 Загальний воксельний об'єм (World Volume)

Загальний світовий воксельний об'єм є центральною структурою даних усієї системи трасування. Він формується шляхом вокселізації та малювання всіх об'єктів сцени у спільну дискретну систему координат. Він представлений у шейдері у вигляді 3D текстури з 3 MIP рівнями, що дозволяє пропускати порожні області і оптимізувати рей трейсинг різних видів освітлення, таких як:

- ◇ Освітлення від сонця з тінями і півтінями;
- ◇ Навколишнє перекриття (*англ.* **Ambient Occlusion, AO**).

Запропонована структура світового воксельного об'єму забезпечує ієрархічну організацію простору, що зменшує обчислювальні витрати під час трасування променів. Використання MIP-рівнів формує багаторівневе представлення сцени, яке дозволяє адаптивно змінювати дискретизацію та скорочувати кількість перевірок у порожніх областях. Такий підхід ефективно скорочує кількість перевірок перетину променів, особливо в великих сценах із численними об'єктами.



(а) Кольорове зображення



(б) Загальний об'єм, MIP 0



(в) Загальний об'єм, MIP 1



(г) Загальний об'єм, MIP 2

Рис. 2.5:
Загальний світовий воксельний об'єм з MIP-рівнями.

2.4 Оптимізації

Для досягнення прийнятної продуктивності у реальному часі необхідно застосовувати низку оптимізацій:

- ◇ використання MIP-рівнів для пропуску порожніх ділянок;
- ◇ обмеження максимальної кількості кроків DDA;
- ◇ ранній вихід при досягненні повністю непрозорого вокселя;
- ◇ зниження точності обчислень там, де це не впливає на візуальну якість (з використанням MIP рівнів для LOD);
- ◇ Представлення найменшої ланки загального воксельного об'єму за допомогою бітів, де 1 — це твердий воксель, а 0 — порожній; це зменшує простір відеопам'яті для збереження об'єму в 8 разів та додає додатковий рівень октодереву для оптимізації швидкодії.

Сукупність наведених методів дозволяє реалізувати ефективний алгоритм трасування променів у воксельному просторі, придатний для використання у інтерактивних графічних застосунках.

3 РЕАЛІЗАЦІЯ АЛГОРИТМУ ТРАСУВАННЯ ПРОМЕНІВ У ВОКСЕЛЬНОМУ ПРОСТОРІ

3.1 Створення базових структур даних та матеріалів на базі рушія Bevy мовою Rust

Для реалізації алгоритму трасування було вибрано ігровий модульний рушій Bevy версії 0.15, написаний мовою програмування Rust для зручного визначення та реалізації графічного пайплайну, і шейдерну мову WGSL для реалізації основних процесів на GPU:

- ◇ базове трасування кольору орієнтованих воксельних об'єктів на сцені на поверхню обмежувальних коробок (`VoxelRayTracedMaterial`);
- ◇ відкладене трасування тіней та навколишнього перекриття за допомогою загального воксельного об'єму та G-буферів (`DeferredRayTracedPipelineOptimized`).

3.1.1 Архітектура системи плагінів матеріалів

В даній роботі ми визначаємо систему воксельних матеріалів та пайплайнів в межах графу рендерингу рушія Bevy (Bevy Render Graph). Для цього ми створюємо певні плагіни та системи, які відповідають за різні аспекти рендерингу воксельних об'єктів та їх інтеграцію з основним графом рендерингу:

1. `VoxelPlugin` — основний плагін, який включає в себе інші підплагіни, які відповідають за різні аспекти графічного застосунку:
 - (a) `Image3dPlugin` — завантаження двовимірних зрізів 3D моделі у

відомих форматах зображення (JPG, PNG тощо) та вивантаження їх у 3D текстури на GPU;

- (б) `WorldVolumePlugin` — ініціалізація та обробка (заповнення, очищення) загального воксельного об'єму;
- (в) `VoxelRayTracedMaterialPlugin` — реєстрація та ініціалізація основного матеріалу для первинного воксельного трасування;
- (г) `RayTracedPostProcessingPlugin` — плагін пост-процесингу, що включає в себе пайплайни відкладеного трасування та очищення G-буфера в кінці проходу рендеру (*англ.* **render pass**);
 - i. `RayTracedAttachmentsPlugin` — менеджмент компонентів G-буфера для відкладеного рендерера.

2. Додаткові плагіни:

- ◇ `MaterialPlugin<DebugVoxelRayTracedMaterial>` — матеріал для візуального зневадження загального об'єму;
- ◇ `ExtractResourcePlugin<DebugOptions>` — плагін для перетворення ж зневаджувальної конфігурації рендерера для роботи в пайплайні.

3.1.2 Організація модульної структури проєкту

Проєкт організовано за принципом модульної архітектури з чіткою розділеністю відповідальності між компонентами. Кореневі модулі `lib.rs` та `main.rs` забезпечують точки входу для бібліотеки та виконавчого файлу відповідно.

Основні модулі проєкту структуровані наступним чином:

- ◇ `materials` — модуль для визначення матеріалів та їх поведінки:
 - `voxel_ray_traced.rs` — імплементація основного матеріалу `VoxelRayTracedMaterial` для первинного трасування;
 - `debug/debug_voxel_ray_traced.rs` — допоміжний матеріал для візуалізації `Voxel World Volume`;
- ◇ `post_processing` — модуль для пост-процесингу та відкладеного рендерингу:

- `ray_traced_attachments.rs` — управління G-buffer текстурами та їх механіка оновлення;
- `deferred_ray_traced.rs` — реалізація вузла відкладеного рендерингу та світлового проходу;
- `clear_ray_traced_resources.rs` — системи для очищення ресурсів між кадрами;

◇ `utils` — модуль утиліт:

- `image_3d.rs` — завантаження та обробка 3D текстур із 2D зрізів;
- `expand.rs` — макрос для створення розширень 3D зображень під час компіляції: “.3d.png”, “.3d.jpg” тощо;
- `world_volume/` — підмодуль для управління глобальним воксельним простором:
 - * `clear.rs` — compute шейдер для очищення світового об’єму;
 - * `fill.rs` — compute шейдер для заповнення світового об’єму воксельними моделями;
 - * `mip_gen.rs` — compute шейдер для генерації мірмар-рівнів ієрархічного трасування;

Така організація дозволяє легко розширювати функціональність проєкту, тестувати окремі компоненти та підтримувати чистоту кодової бази.

3.1.3 Реалізація структури `VoxelRayTracedMaterial`

Для забезпечення інтеграції воксельного трасування з графічним пайплайном `Bevy` було створено спеціалізацію трейту `Material`. Трейт `Material` в `Bevy` визначає інтерфейс для матеріалів, які можуть бути застосовані до 3D об’єктів, та дозволяє налаштовувати їх рендеринг на рівні GPU.

Структура `VoxelRayTracedMaterial` визначена наступним чином:

```

#[derive(Asset, TypePath, AsBindGroup, Debug, Clone, Default)]
pub struct VoxelRayTracedMaterial {
    #[texture(0, dimension = "3d", sample_type = "u_int")]
    pub voxels: Handle<Image>,

    #[storage_texture(1, access = ReadWrite, image_format =
R32Float, visibility(fragment))]
    pub depth_texture: Option<Handle<Image>>,

    #[storage_texture(2, access = WriteOnly, image_format =
Rgba32Float, visibility(fragment))]
    pub normal_texture: Option<Handle<Image>>,

    pub alpha_mode: AlphaMode,
}

```

Дана структура використовує макрос `AsBindGroup`, який автоматично генерує код для зв'язування ресурсів з GPU. Кожне поле структури анотоване атрибутами, що визначають спосіб його передачі до шейдера:

- ◇ `voxels` — 3D текстура з цілочисельним семплуванням, що містить воксельні дані моделі (binding 0);
- ◇ `depth_texture` — мапа глибини G-буфера з можливістю читання та запису у фрагментному шейдері (binding 1);
- ◇ `normal_texture` — мапа нормалей G-буфера з можливістю запису (binding 2);
- ◇ `alpha_mode` — режим обробки прозорості для інтеграції з існуючим пайплайном рендерингу.

Імплементация трейту `Material` визначає спеціалізовану поведінку рендерингу:

```

impl Material for VoxelRayTracedMaterial { ... }

```

Плагін `VoxelRayTracedMaterialPlugin` реєструє матеріал в системі Bevy та додає системи для автоматичного управління розмірами воксельних об'ємів.

3.1.4 Система компонентів для воксельних об'єктів

Для коректного відображення воксельних моделей необхідно автоматично адаптувати їх геометричне представлення до фактичних розмірів воксельного об'єму. Для цього введено компонент `VoxelVolumeSize`, який зберігає тривимірні розміри воксельного простору:

```
#[derive(Default, Component, Debug, Clone, ...)]
pub struct VoxelVolumeSize(Extent3d);
```

Компонент є обгорткою навколо структури `Extent3d`, яка містить ширину, висоту та глибину текстури у вокселях. Система `apply_voxel_volume_size` автоматично визначає розміри воксельного об'єму з дескриптора 3D текстури та додає відповідний компонент до сутності ECS (*англ.* **Entity**). А система `set_voxel_volume_mesh` динамічно оновлює меш-геометрію відповідно до розмірів воксельного об'єму. Вона спрацьовує лише при додаванні компонента `VoxelVolumeSize` до сутності (перевірка `size.is_added()`). Вона виконує наступні операції:

1. Обчислює фізичні розміри об'єму у світових координатах, множачи кількість вокселів на константу `VOXEL_SIZE`;
2. Створює нову кубічну геометрію (`Cuboid`) з визначеними розмірами;
3. Налаштовує обмежувальну коробку `Aabb` (*англ.* **Axis-Aligned Bounding Box**) для коректного frustum culling — відсікання об'єктів поза полем зору камери.

3.1.5 Система World Volume

Системи ініціалізації та обробки світового об'єму визначена в плагіні `WorldVolumePlugin` і складається з таких компонентів:

- ◇ `ExtractResourcePlugin`-и для роботи з ресурсами світового об'єму в пайплайнах;
- ◇ `FillWorldVolume` — вузол і пайплайн для заповнення світового об'єма

шляхом інверсивного мапування орієнтованих воксельних об'ємів у нього;

- ◇ **ClearWorldVolume** — вузол і пайплайн для очищення світового об'єму між кадрами;
- ◇ **MipGenWorldVolume** — генерація MIP рівнів для ієрархічного трасування та оптимізації вибору рівня деталізації під час рендерингу;
- ◇ Функція екстракції даних у **finish** — синхронізація колекції воксельних об'ємів між основним та рендер-світом з обчисленням інверсних матриць трансформації для переходу між координатними системами, що використовуються пізніше в **FillWorldVolume**.

3.1.6 Завантаження воксельних даних з формату 2D слайсів

Завантаження воксельних моделей відбуваються з файлів 2D зображень, експортованих з **MagicaVoxel**, що містять двовимірні зрізи воксельних об'ємів. Це відбувається асинхронно за допомогою користувачького **AssetLoader**, що завантажує зображення, ділить його на шари і зберігає у вигляді вбудованого типу **Image**. Ми обчислюємо кожен наступний піксель, що передається в 1D-масив, за допомогою наступної простої формули:

$$P(x,y,c,r) = \begin{pmatrix} x + c \times w \\ y + r \times h \end{pmatrix},$$

де c та r — це поточні стовпець та рядок відповідно, x та y — поточні координати плитки, а w та h — ширина та висота однієї плитки.[8]

3.2 Реалізація повного пайплайну відкладеного рендерингу для трасування у воксельному просторі

3.2.1 Теоретичні засади відкладеного рендерингу

Відкладений рендеринг заснований на тому, що ми переносимо (відкладаємо) обчислення світла, як важке обчислення, на пізніший етап, коли замість вираховування світла (в нашому випадку трасуванням променів) для кожного об'єкту окремо, ми робимо це для всього екрану в цілому. Відкладене затінення включає два проходи:

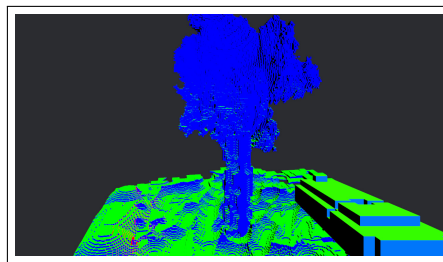
1. на першому проході, який називається проходом геометрії, ми малюємо сцену один раз та отримуємо всі геометричні дані зі сцени, які ми зберігаємо G-буфері; в нашому випадку це глибина сцени та нормалі всіх об'єктів;
2. на другому проході ми використовуємо дані із G-буфера та додаткові GPU-ресурси (наприклад, World Volume), для рендерингу затінення та навколишнього перекриття як ефект пост-процесингу, оскільки рендеринг відбувається на весь екран. [9]



(а) Альбедо (колір без тіней)



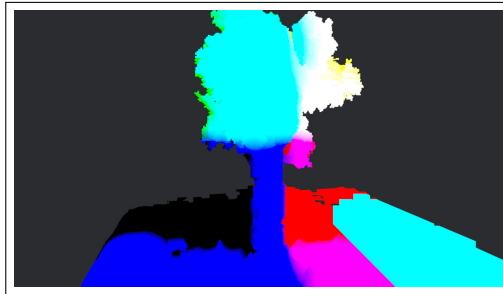
(б) Глибина сцени



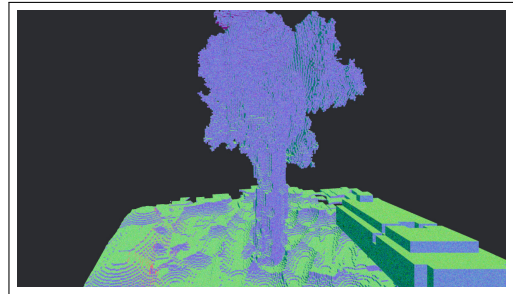
(в) Нормалі поверхні

Рис. 3.1:
Проміжні мапи у G-буфері для невеликої сцени.

Оскільки у нас буде відбуватися трасування променів у відкладеному шейдері, то з проміжних мап G-буфера ми створимо похідні мапи, характерні саме для трасування в екранному просторі (*англ.* **Screen-Space Ray Tracing, SSRT**) — мапу початку і напрямку початкового променя:



(а) Початкові точки променя; колір позначає розташування у просторі $RGB \rightarrow XYZ$;



(б) Напрямки променя; колір позначає напрямок у просторі (є похідним від нормалей поверхні)

3.2.2 Створення G-buffer attachments

Для управління текстурами G-буфера та забезпечення їх автоматичного масштабування при зміні розміру вікна було створено компонент **RayTracedAttachments**. Цей компонент прикріплюється до камери та зберігає ключі/ідентифікатори (*англ.* **handle**) всіх необхідних текстур для відкладеного рендерингу:

```
[derive(Component, Clone, ExtractComponent)]
pub struct RayTracedAttachments {
    current_size: UVec2,
    pub normals: Handle<Image>,
    pub depth: Handle<Image>,
    pub blue_noise: Handle<Image>,
}
```

Поля структури мають наступне призначення:

- ◇ **current_size** — поточний розмір viewport камери у пікселях для відстеження необхідності перестворення текстур;
- ◇ **normals** — ключ мапи нормалей, що заповнюється в проході геометрії;
- ◇ **depth** — ключ мапи глибини;

- ◇ **blue_noise** — текстура блакитного шуму для покращення якості стохастичних ефектів, таких як ambient occlusion та м'які тіні. Вона завантажується з файлу, *однак не використовується в процесі рендерингу, і **можже** бути використана для візуального покращення в майбутньому.*

Система **add_attachments** автоматично створює та прикріплює компонент **RayTracedAttachments** до камер з маркером **RayTracedPostProcessing**, які ще не мають цього компонента. Вона виконує наступні дії:

1. Визначає фізичний розмір viewport камери або використовує значення за замовчуванням 1280×720 ;
2. Створює текстури глибини та нормалей з відповідними форматами та прапорцями використання (**STORAGE_BINDING** для запису та читання з фрагментних шейдерів);
3. Прикріплює компонент **RayTracedAttachments** та **Jitter** до сутності камери;
4. Оновлює всі існуючі матеріали **VoxelRayTracedMaterial**, встановлюючи їм посилання на створені текстури глибини та нормалей.

Система **resize_attachments** відстежує зміни розміру вікна та динамічно перестворює текстури G-буфера. При виявленні зміни розміру viewport вона:

1. Видаляє старі текстури з пам'яті GPU для уникнення витоку ресурсів;
2. Створює нові текстури з оновленими розмірами;
3. Оновлює ключі текстур в компоненті **RayTracedAttachments**;
4. Проходить по всіх матеріалах **VoxelRayTracedMaterial** та замінює старі хендли текстур на нові.

Система **update_jitter** оновлює компонент **Jitter** для кожної камери, що містить його, генеруючи нові випадкові значення для кожного кадру. Це забезпечує варіативність стохастичних ефектів рей трейсингу і дозволяє використати вбудований метод темпорального згладжування (*англ.* **Temporal anti-aliasing, TAA**).

3.3 Написання алгоритму трасування шейдерною мовою WGSL

ВИСНОВКИ

СПИСОК ЛІТЕРАТУРИ

- [1] Wikipedia. Voxel — Wikipedia, The Free Encyclopedia. — 2026. — [Доступ 10-03-2026]. URL: <http://en.wikipedia.org/wiki/Voxel>.
- [2] Laine Samuli, Karras Tero. Efficient sparse voxel octrees // Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games. — I3D '10. — New York, NY, USA : Association for Computing Machinery, 2010. — P. 55–63. — URL: <https://doi.org/10.1145/1730804.1730814>.
- [3] Shirley Peter. Ray Tracing in One Weekend. — 3.0.2 edition. — 2020. — [Доступ 10-03-2026]. URL: raytracing.github.io/books/RayTracingInOneWeekend.html.
- [4] Yagel R., Cohen D., Kaufman A. Discrete ray tracing // IEEE Computer Graphics and Applications. — 1992. — Vol. 12, no. 5. — P. 19–28.
- [5] Beier Thaddeus, Neely Shawn. Feature-based image metamorphosis // Proceedings of the 19th Annual Conference on Computer Graphics and Interactive Techniques. — SIGGRAPH '92. — New York, NY, USA : Association for Computing Machinery, 1992. — P. 35–42. — URL: <https://doi.org/10.1145/133994.134003>.
- [6] Wikipedia. Digital differential analyzer (graphics algorithm) — Wikipedia, The Free Encyclopedia. — 2026. — [Доступ 10-03-2026]. URL: [http://en.wikipedia.org/wiki/Digital differential analyzer \(graphics algorithm\)](http://en.wikipedia.org/wiki/Digital_differential_analyzer_(graphics_algorithm)).
- [7] Amanatides John, Woo Andrew. A Fast Voxel Traversal Algorithm for Ray Tracing // Proceedings of EuroGraphics. — 1987. — 08. — Vol. 87.
- [8] O. ГИТОВ. Loading 3D texture in Bevy. — <https://konceptosociala.eu.org/post/loading-3d-texture-in-bevy>. — 2025. — [Доступ 10-03-2026].

- [9] Joey DeVries. LearnOpenGL - Deferred Shading — learnopengl.com. — <https://learnopengl.com/Advanced-Lighting/Deferred-Shading>. — [Доступ 10-03-2026].

ДОДАТКИ



Рис. 3.3: Два котика